TP3 : VHDL Séquentiel, comptage du temps

Jaime GOMES-PARENTE et Robin BRICHE

Introduction

L'objectif principal de ce TP est de compléter notre étude des machines à états en les appliquant au comptage du temps, avec ou sans prise en compte d'entrées extérieures.

Au cours de ce TP, nous avons d'abord étudié l'implémentation d'un diviseur de fréquence pour adapter l'horloge de la carte FPGA. Ensuite, nous avons modélisé et implémenté le contrôleur d'un carrefour à feux de circulation selon une séquence temporelle stricte, puis nous l'avons fait évoluer pour prendre en compte des capteurs de véhicules.

Exercice 1 : Diviseur de fréquence

2. Expliquez succinctement le fonctionnement du diviseur de fréquence.

Le diviseur de fréquence permet de réduire la fréquence du signal d'horloge d'entrée “clock_in”. Il utilise un compteur interne “c” qui s'incrémente à chaque front montant jusqu'à atteindre la valeur de divisor -1. Le signal de sortie “clock_out” est à l'état '1' lorsque c < divisor / 2 et à '0' autrement, garantissant ainsi un rapport cyclique de 50 %.

3. D’après la fréquence d’entrée et la valeur du diviseur dans l’exemple, quelle est la fréquence du signal de sortie ?

La carte est équipée d'un générateur d'horloge de 100 MHz. Dans l'exemple d'instanciation, le paramètre divisor est défini à 20 000 000.
La fréquence du signal de sortie est donc : 100 000 000 / 20 000 000 = 5 Hz.

4. Programmez le FPGA et vérifiez le fonctionnement du diviseur.

Lors de la programmation, on constate que la “LED_0” clignote rapidement, ce qui valide le fonctionnement matériel du diviseur.
Si l'on maintient le bouton “BTN_C” appuyé, la LED s'éteint car le reset asynchrone maintient la sortie à 0.

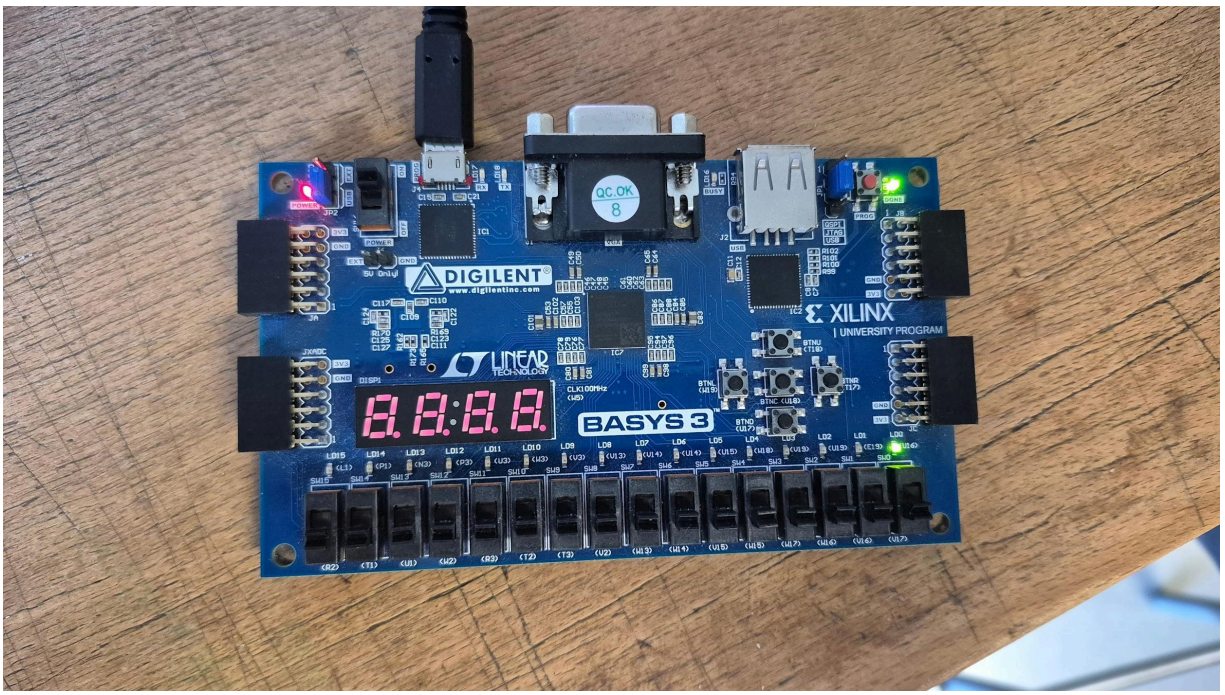


Figure 1.1 LED qui clignote quand on ne touche à rien

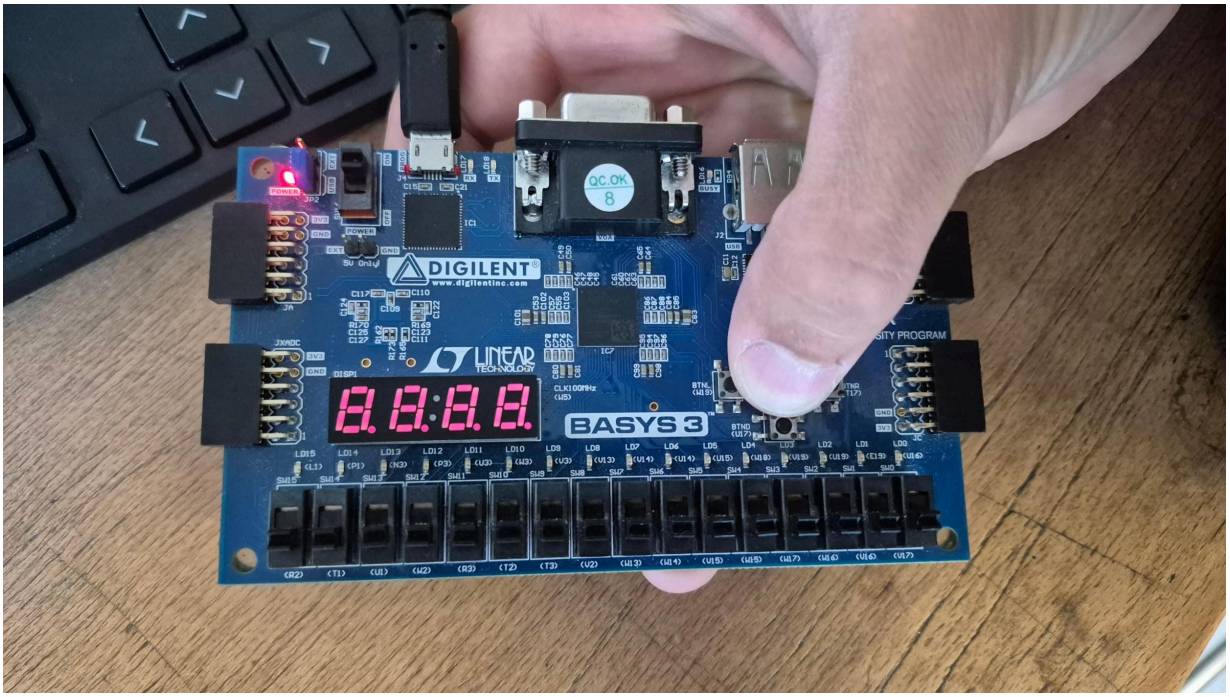


Figure 1.2 LED qui ne clignote plus quand le bouton est appuyé

Exercice 2 : Feux de circulation

1. Définissez la durée de toutes les phases et le déroulement de la séquence.

Les deux axes du carrefour fonctionnent de manière symétrique. La phase rouge = 10 s et la phase orange = 2 s. D'où un Feu doit rester rouge tout le long du cycle de l'autre feu d'où Phase Verte = 10 – 2 = 8 s . La séquence complète de 20 secondes est la suivante :

- Phase 1 (8s) : Axe 1 Vert / Axe 2 Rouge
- Phase 2 (2s) : Axe 1 Orange / Axe 2 Rouge
- Phase 3 (8s) : Axe 1 Rouge / Axe 2 Vert
- Phase 4 (2s) : Axe 1 Rouge / Axe 2 Orange

2. En fonction des durées des phases, quelle devrait être la fréquence minimale de l'horloge ?

Les durées étant de 2s et 8s, le plus grand commun diviseur est de 2 secondes. La fréquence minimale serait de 0,5 Hz. Mais, pour une logique de comptage plus simple, une horloge divisée à 1 Hz est privilégiée.

3. Modélisez le contrôleur en VHDL. Pour générer le signal d'horloge approprié, vous instancierez un diviseur de fréquence comme dans l'exemple de l'exercice 1.

Le système est modélisé sous la forme d'une machine à états rythmée par un signal d'horloge de 1 Hz.

Le code instancie d'abord un composant clock_divider qui prend l'horloge d'entrée CLK à 100 MHz et génère le signal lent clk_1hz. Le process réagit quand clock_1Hz a un front montant. Il évalue un état courant grâce au signal etat_courant et assigne les vecteurs de bits correspondants aux sorties LED_3_0 (axe 1) et LED_7_4 (axe 2) afin d'allumer les feux appropriés (Etat 0 : Axe 1 Vert donc LED_3_0 = « 0001 » et Axe 2 rouge donc LED_7_4 = « 0100 » etc).

Un signal timer est incrémenté à chaque coup d'horloge. Lorsque le timer atteint la limite fixée pour la phase en cours (7 pour les 8 secondes du vert, 1 pour les 2 secondes de l'orange), le système bascule vers l'état suivant et remet le timer à zéro, créant ainsi une boucle continue.

```
entity controleur_feux is
    port (
        CLK     : in bit;
        BTN_C   : in bit;
        LED_3_0 : out bit_vector(3 downto 0);
        LED_7_4 : out bit_vector(3 downto 0)
    );
end controleur_feux;


architecture Behavioral of controleur_feux is


    component clock_divider
        generic(divisor : integer);
        port(
            clock_in  : in bit;
            reset     : in bit;
            clock_out : out bit
        );
    end component;


    signal clk_1Hz : bit;


    type state_type is (PHASE_1, PHASE_2, PHASE_3, PHASE_4);
    signal etat_courant : state_type := PHASE_1;

    signal compteur : integer range 0 to 10 := 0;
```

```
begin


    diviseur_inst : clock_divider
        generic map (divisor => 100000000)
        port map (
            clock_in  => CLK,
            reset     => '0',
            clock_out => clk_1Hz
        );


    process(clk_1Hz, BTN_C)
    begin
        if BTN_C = '1' then
            etat_courant <= PHASE_1;
            compteur <= 0;

        elsif clk_1Hz'event and clk_1Hz = '1' then
            compteur <= compteur + 1;

            case etat_courant is
                when PHASE_1 =>
                    if compteur >= 7 then
                        etat_courant <= PHASE_2;
                        compteur <= 0;
                    end if;

                when PHASE_2 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_3;
                        compteur <= 0;
                    end if;

                when PHASE_3 =>
                    if compteur >= 7 then
                        etat_courant <= PHASE_4;
                        compteur <= 0;
                    end if;

                when PHASE_4 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_1;
                        compteur <= 0;
                    end if;
            end case;
        end if;
    end process;

   process(etat_courant)
   begin
       case etat_courant is
           when PHASE_1 =>
               LED_3_0 <= "0001";
               LED_7_4 <= "0100";

           when PHASE_2 =>
               LED_3_0 <= "0010";
               LED_7_4 <= "0100";

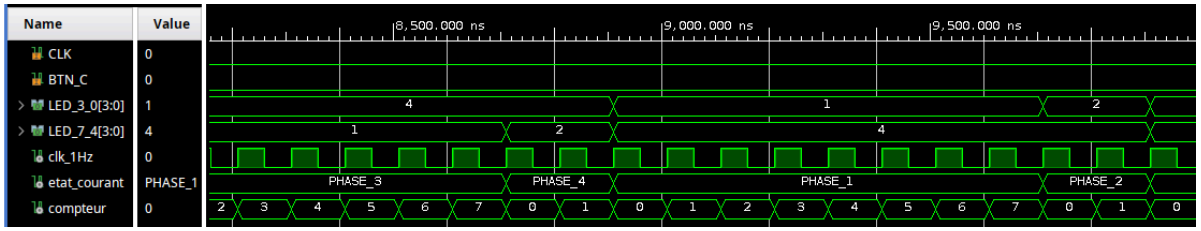
           when PHASE_3 =>
               LED_3_0 <= "0100";
               LED_7_4 <= "0001";

           when PHASE_4 =>
               LED_3_0 <= "0100";
               LED_7_4 <= "0010";
       end case;
   end process;

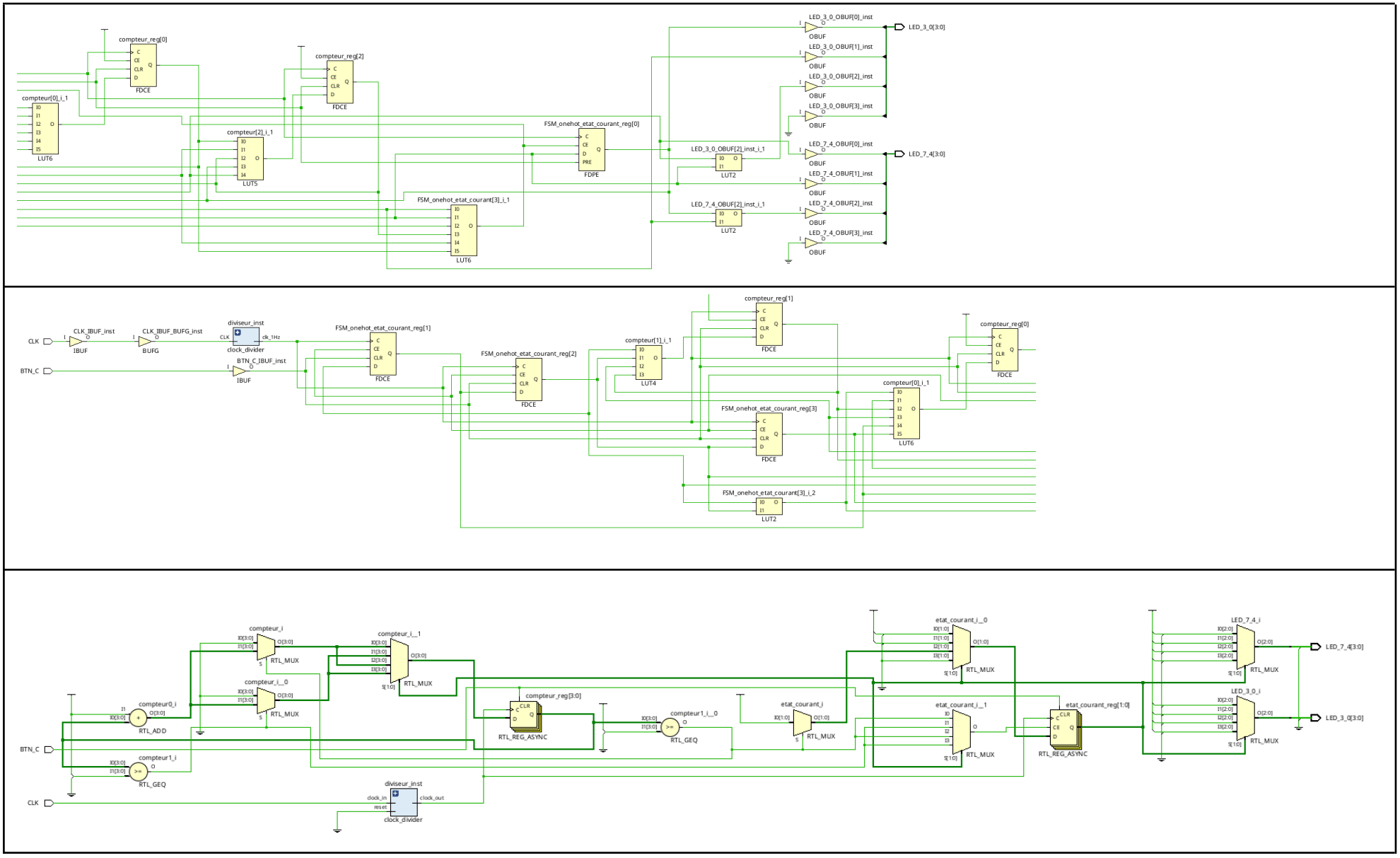

end Behavioral;
```

4. Testez votre modèle en simulation. Vous pouvez directement forcer le signal de sortie du diviseur pour générer l'horloge d'entrée de votre système, inutile de forcer le signal CLK avec une horloge à 100 MHz.

On observe bien les changements d'état des bus de sortie LED_3_0 et LED_7_4 à des intervalles réguliers et le timer qui s'incrémente à chaque front montant de clock_1hz. Le feu vert ("001") de l'axe 1 reste actif pendant 8 périodes d'horloge (8s), avant de passer à l'orange ("010") pour 2 périodes d'horloge (2s). Pendant ces 10 s, l'axe 2 reste au rouge ("100"). Ensuite, la symétrie s'opère et les rôles s'inversent. (Dans le test on a direct force clock le clock_1hz pour que le test soit plus simple à visualiser)



Comme demandé on met aussi le schéma pré-synthèse et post synthèse, 1ere image pour la présynthèse et les deux suivantes pour le post synthèse.


compteur_reg[0]
FDCE
compteur[0]_i_1
LUT6
compteur_reg[2]
FDCE
compteur[2]_i_1
LUT5
FSM_onehot_etat_courant_reg[0]
FDPE
FSM_onehot_etat_courant[3]_i_1
LUT6
LED_3_0_OBUF[0]_inst
OBUF
LED_3_0_OBUF[1]_inst
OBUF
LED_3_0_OBUF[2]_inst
OBUF
LED_3_0_OBUF[3]_inst
OBUF
LED_3_0[3:0]
LED_3_0_OBUF[2]_inst_i_1
LUT2
LED_7_4_OBUF[0]_inst
OBUF
LED_7_4_OBUF[1]_inst
OBUF
LED_7_4_OBUF[2]_inst
OBUF
LED_7_4_OBUF[3]_inst
OBUF
LED_7_4[3:0]
LED_7_4_OBUF[2]_inst_i_1
LUT2
CLK
CLK_IBUF_inst
IBUF
CLK_IBUF_BUFG_inst
BUFG
diviseur_inst
clock_divider
clk_1Hz
BTN_C
BTN_C_IBUF_inst
IBUF
FSM_onehot_etat_courant_reg[1]
FDCE
FSM_onehot_etat_courant_reg[2]
FDCE
compteur[1]_i_1
LUT4
compteur_reg[1]
FDCE
compteur_reg[0]
FDCE
compteur[0]_i_1
LUT6
FSM_onehot_etat_courant_reg[3]
FDCE
FSM_onehot_etat_courant[3]_i_2
LUT2
BTN_C
CLK
compteur_i
RTL_MUX
compteur_i__1
RTL_MUX
compteur_i__0
RTL_MUX
compteur0_i
RTL_ADD
compteur1_i
RTL_GEQ
compteur_reg[3:0]
RTL_REG_ASYNC
compteur1_i__0
RTL_GEQ
diviseur_inst
clock_divider
etat_courant_i__0
RTL_MUX
etat_courant_i
RTL_MUX
etat_courant_i__1
RTL_MUX
etat_courant_reg[1:0]
RTL_REG_ASYNC
LED_7_4_i
RTL_MUX
LED_7_4[3:0]
LED_3_0_i
RTL_MUX
LED_3_0[3:0]

5. Programmez le FPGA et vérifiez le fonctionnement.

Après synthèse, implémentation et génération du bitstream, on téléverse le programme sur la carte. Sur celle ci, on peut alors directement observer que la séquence s'éxecute automatiquement en boucle fermée. Les LEDs associées à l'axe 1 (LED_0 à LED_2) et celles associées à l'axe 2 (LED_4 à LED_6) altèrnent conformément aux temporisation réelles : 8 secondes de vert, 2 secondes d'orange et 10 secondes de rouge.

On a donc 4 phasassees; la Phase 1 allume la LED 0 (vert axe 1) et la LED 6 (rouge axe 2) pendant 8 secondes (Figure 2.1), suivie de la Phase 2 avec la LED 1 (orange axe 1) et la LED 6 pendant 2 secondes (Figure 2.3), puis de la Phase 3 activant la LED 2 (rouge axe 1) et la LED 4 (vert axe 2) pendant 8 secondes (Figure 2.2), et enfin la Phase 4 allumant la LED 2 et la LED 5 (orange axe 2) pendant 2 secondes (Figure 2.4)

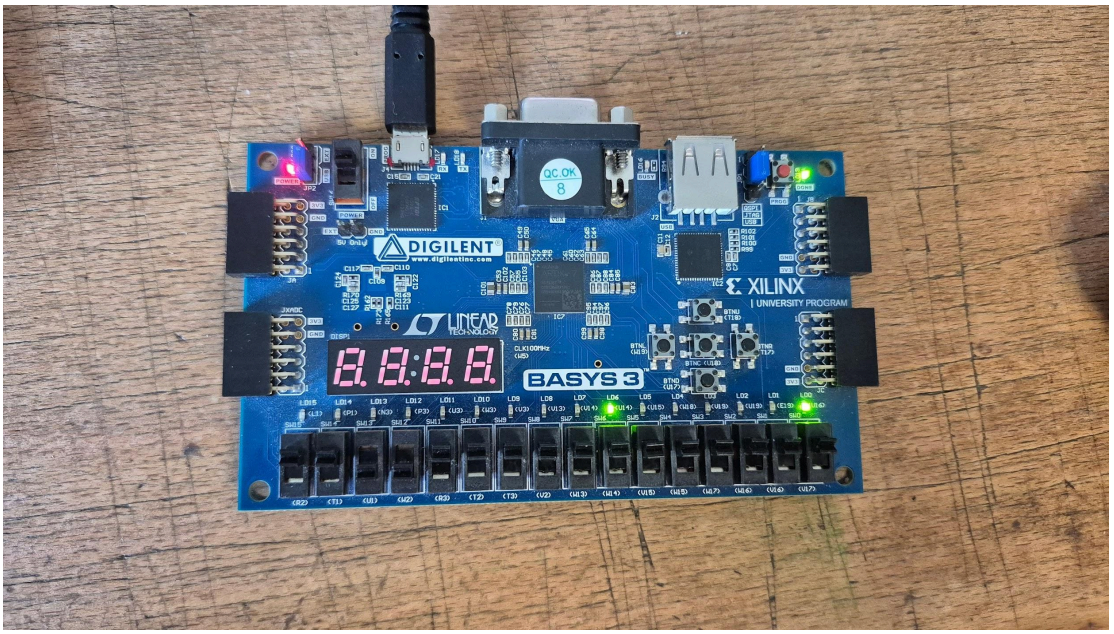


Figure 2.1 : LED_0 et LED_6, Axe 1 vert, Axe 2 rouge, 8 secondes

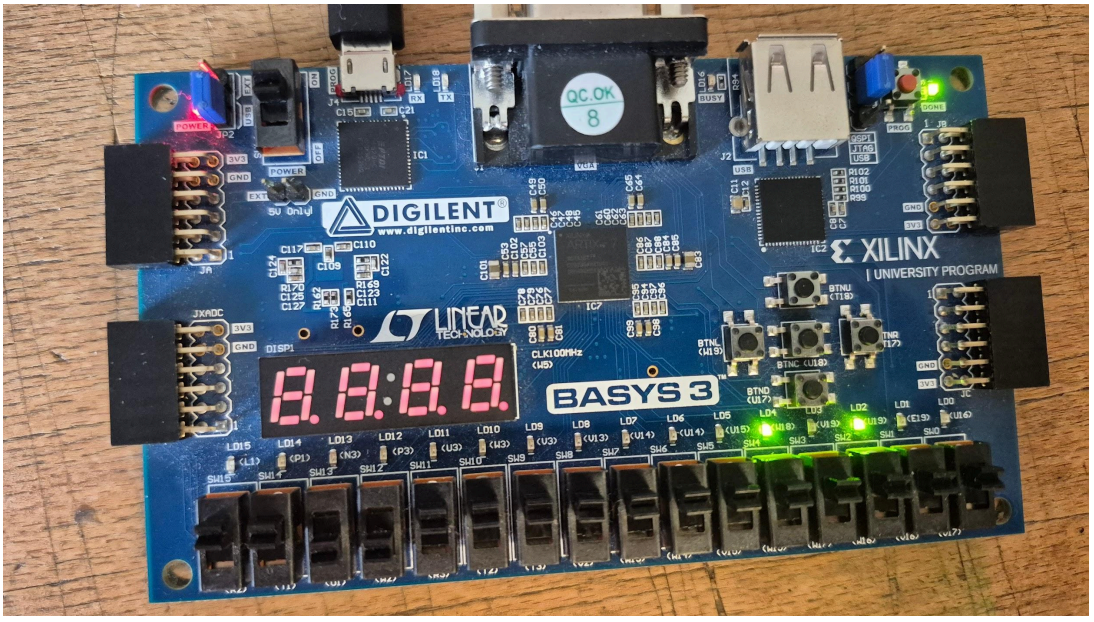


Figure 2.2 : LED_2 et LED_4, Axe 1 rouge, Axe 2 vert, 8 secondes

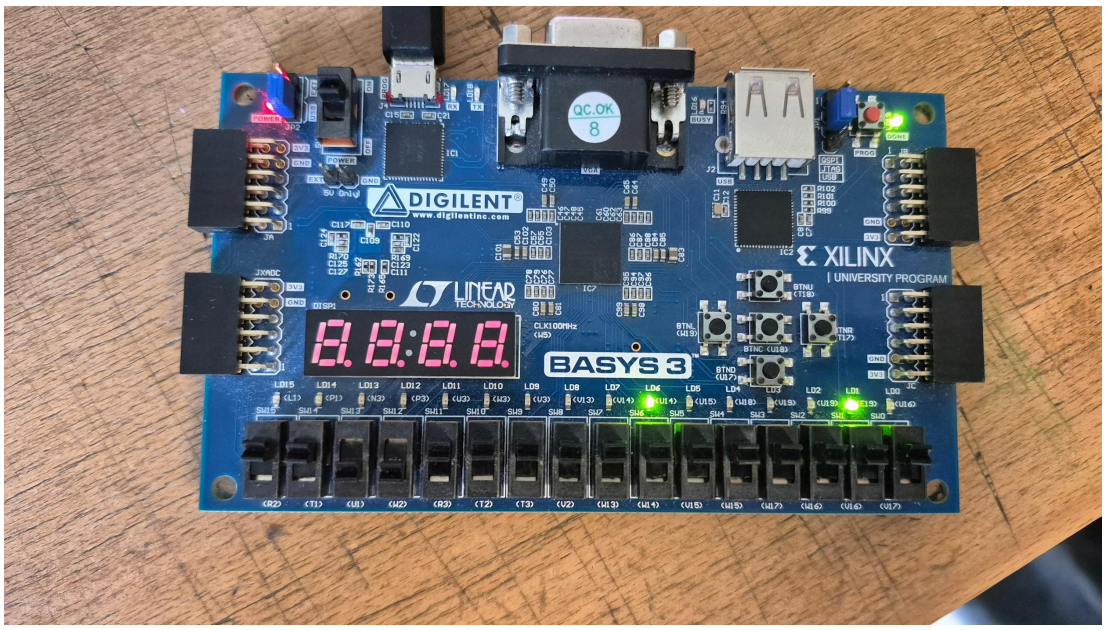


Figure 2.3 : LED_1 et LED_6, Axe 1 orange, Axe 2 rouge, 2 secondes

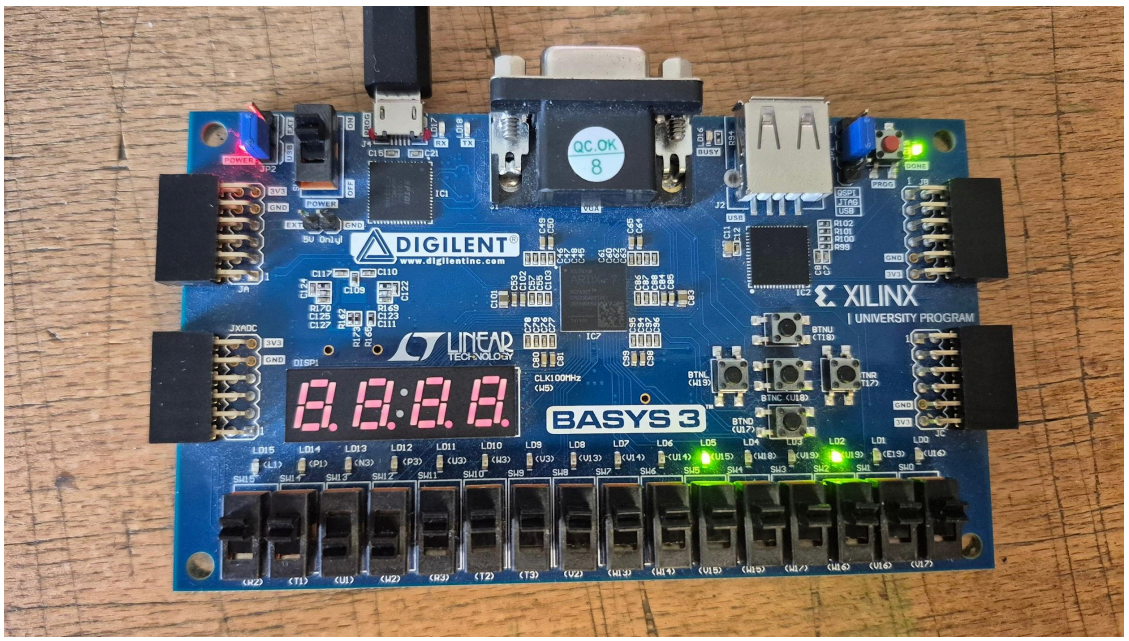


Figure 2.4: LED_2 et LED_5, Axe 1 rouge, Axe 2 orange, 2 secondes

6. Comment peut-on intégrer une remise à zéro fiable du système ?

Afin de garantir un fonctionnement déterministe et fiable, il faut intégrer au système un reset synchrone; On prend comme signal de reset BTN_C, qui, lorsqu'il est activé au front montant de l'horloge, va forcer l'état initial de la machine et réinitialiser le compteur temporel à 0.

7. Codez, simulez, programmez le FPGA et testez votre solution.

Le code VHDL a été mis à jour en intégrant la condition prioritaire " if BTN_C = '1' then" directement au sein du bloc synchronisé sur "clk_1Hz'event and clk_1Hz ='1'" :

```
entity controleur_feux is
    port (
        CLK     : in bit;
        BTN_C   : in bit;
        LED_3_0 : out bit_vector(3 downto 0);
        LED_7_4 : out bit_vector(3 downto 0)
    );
end controleur_feux;


architecture Behavioral of controleur_feux is

    component clock_divider
        generic(divisor : integer);
        port(
            clock_in : in bit;
            reset : in bit;
            clock_out : out bit
        );
    end component;


    signal clk_1Hz : bit;


    type state_type is (PHASE_1, PHASE_2, PHASE_3, PHASE_4);
    signal etat_courant : state_type := PHASE_1;


    signal compteur : integer range 0 to 10 := 0;


begin



    diviseur_inst : clock_divider
        generic map (divisor => 100000000)
        port map (
            clock_in => CLK,
            reset => '0',
            clock_out => clk_1Hz
        );

    process(clk_1Hz)
    begin
        if clk_1Hz'event and clk_1Hz = '1' then
            if BTN_C = '1' then
                etat_courant <= PHASE_1;
                compteur <= 0;
            else
                compteur <= compteur + 1;
```

```
process(clk_1Hz)
begin
    if clk_1Hz'event and clk_1Hz = '1' then
        if BTN_C = '1' then
            etat_courant <= PHASE_1;
            compteur <= 0;
        else
            compteur <= compteur + 1;

            case etat_courant is
                when PHASE_1 =>
                    if compteur >= 7 then
                        etat_courant <= PHASE_2;
                        compteur <= 0;
                    end if;

                when PHASE_2 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_3;
                        compteur <= 0;
                    end if;

                when PHASE_3 =>
                    if compteur >= 7 then
                        etat_courant <= PHASE_4;
                        compteur <= 0;
                    end if;

                when PHASE_4 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_1;
                        compteur <= 0;
                    end if;
            end case;
        end if;
    end if;
end process;

process(etat_courant)
begin
    case etat_courant is
        when PHASE_1 =>
            LED_3_0 <= "0001";
            LED_7_4 <= "0100";
        when PHASE_2 =>
            LED_3_0 <= "0010";
            LED_7_4 <= "0100";
        when PHASE_3 =>
            LED_3_0 <= "0100";
            LED_7_4 <= "0001";
        when PHASE_4 =>
            LED_3_0 <= "0100";
            LED_7_4 <= "0010";
    end case;
end process;
```

Figure 2.5 : Code VHDL avec prise en compte du reset

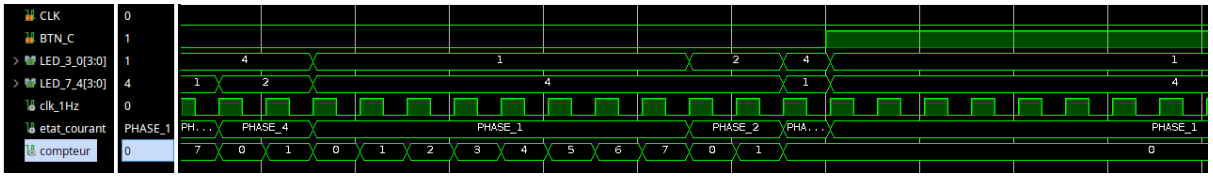


Figure 2. 6 : Simulation avec bouton reset

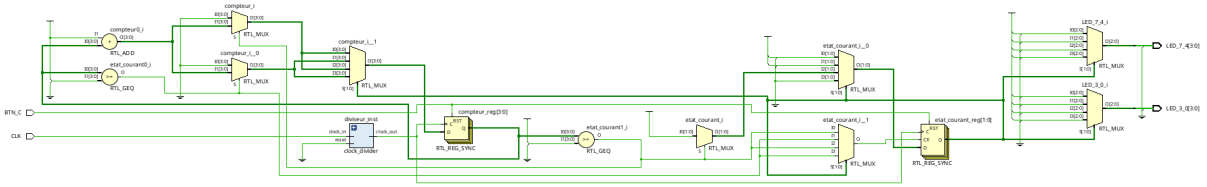


Figure 2.7 : Schéma RTL pré-synthèse

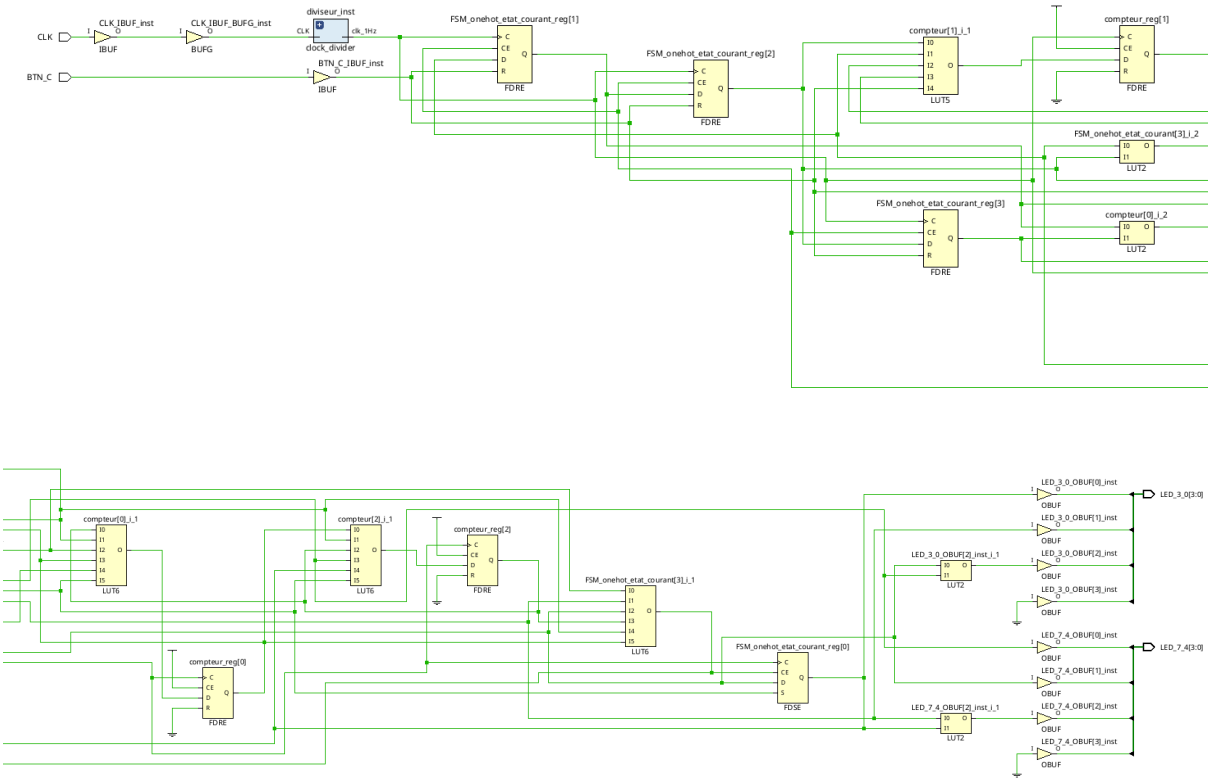


Figure 2.8 : Schéma de synthèse

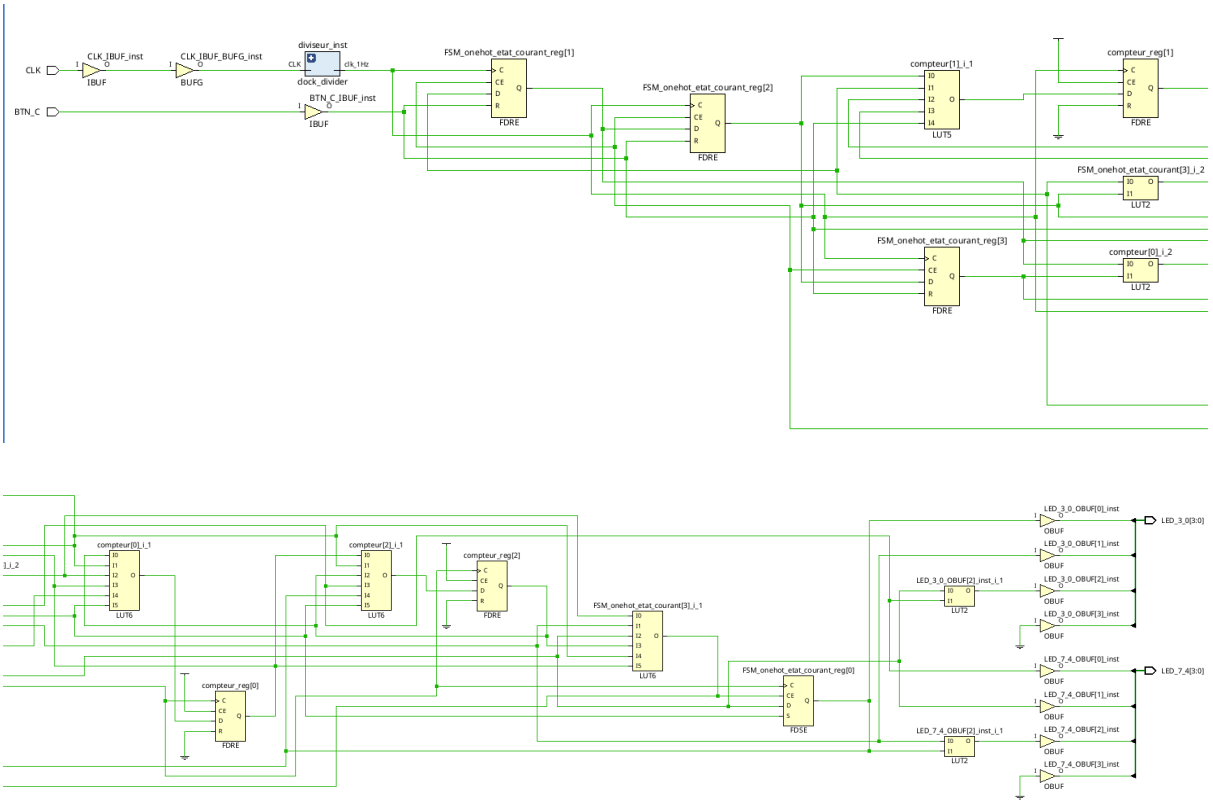


Figure 2.9 : Schéma d'implémentation

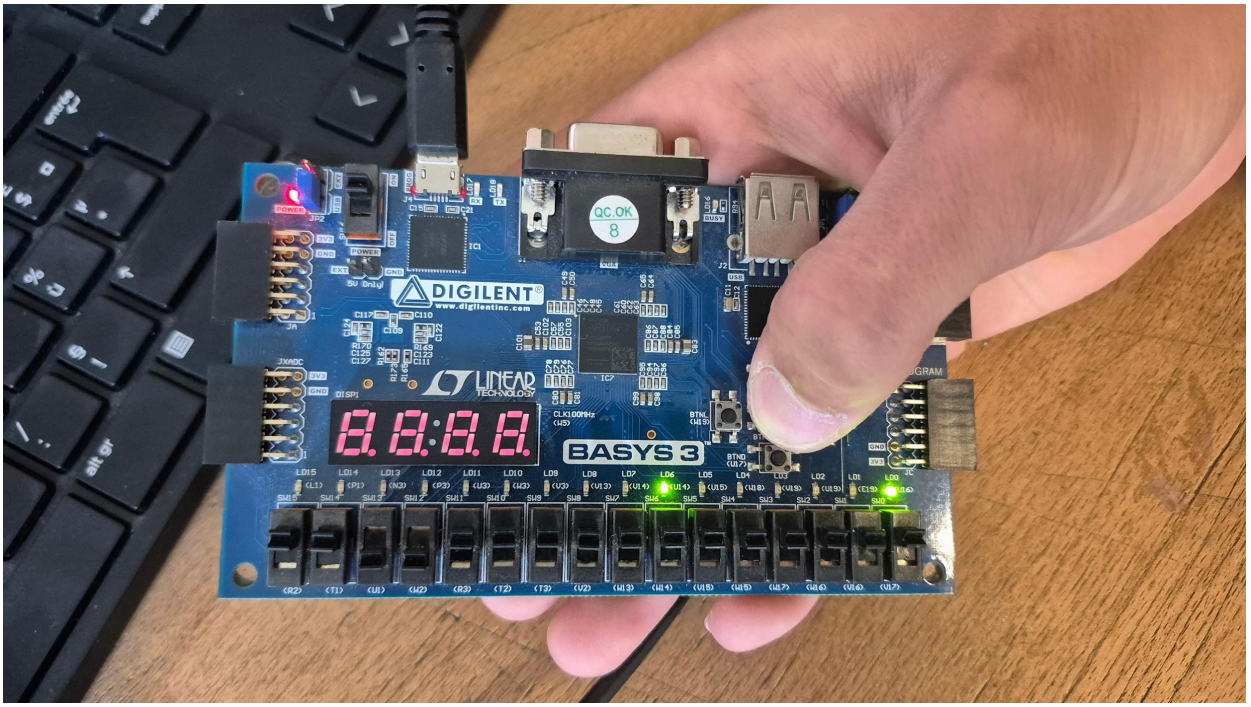


Figure 2.10 : Test du FPGA

Après génération du flux binaire et téléversement sur le FPGA, les tests physiques confirment l'efficacité du dispositif : un appui maintenu sur le bouton poussoir BTN_C fige instantanément l'affichage sur la Phase 1 (LED 0 et LED 6 allumées) et remet le décompte temporel à zéro. En raison de la cadence de l'horloge divisée à 1 Hz, la prise en compte du bouton s'opère au front montant suivant, ce qui valide la nature synchrone du circuit sans perturber le cycle séquentiel lors du relâchement.

Exercice 3 : Prise en compte d'un capteur de voiture

1. Définissez le déroulement de la nouvelle séquence.

Par rapport à l'exercice précédent, la séquence devient asservie à deux boutons, BTN_G pour l'axe 1 et BTN_H pour l'axe 2. La première phase dure minimum 8 secondes puis reste au vert tant qu'aucune voiture n'arrive sur l'axe 2. Dès que BTN_H = '1' après au moins 8 secondes, on passe en Phase 2 (2s en orange) puis en Phase 3 (axe 2 vert, au moins 8 secondes). Enfin, dès qu'une voiture est captée sur l'axe 1, c'est à dire BTN_G = '1', ce qui déclenche la Phase 4 (2s en orange) puis un retour sur la Phase 1.

2. Modifiez votre VHDL en conséquence, sans prise en compte du reset.

Pour être en accord avec le déroulement de la nouvelle séquence que l'on a défini sans la question précédente, il faut ajouter deux entrées, les boutons poussoirs BTN_G et BTN_H. Dans le process, on incrémente le compteur jusqu'à 7 pour garantir les 8 secondes minimum, puis on conditionne le passage à l'état suivant par BTN_H = '1' en Phase 1 et BTN_G = '1' en Phase 3. Les autres phases restent à 2 secondes fixes.

```
entity controleur_feux_capteurs is
    port (
        CLK     : in bit;
        BTN_G   : in bit;
        BTN_H   : in bit;
        LED_3_0 : out bit_vector(3 downto 0);
        LED_7_4 : out bit_vector(3 downto 0)
    );
end controleur_feux_capteurs;
```

```
architecture Behavioral of controleur_feux_capteurs is
    component clock_divider
        generic(divisor : integer);
        port(clock_in, reset : in bit; clock_out : out bit);
    end component;

    signal clk_1Hz : bit;
    type state_type is (PHASE_1, PHASE_2, PHASE_3, PHASE_4);
    signal etat_courant : state_type := PHASE_1;
    signal compteur : integer range 0 to 10 := 0;

begin
    diviseur_inst : clock_divider
        generic map (divisor => 100000000)
        port map (clock_in => CLK, reset => '0', clock_out => clk_1Hz);

    process(clk_1Hz)
    begin
        if clk_1Hz'event and clk_1Hz = '1' then
            case etat_courant is
                when PHASE_1 =>
                    if compteur < 7 then
                        compteur <= compteur + 1;
                    elsif BTN_H = '1' then
                        etat_courant <= PHASE_2;
                        compteur <= 0;
                    end if;

                when PHASE_2 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_3;
                        compteur <= 0;
                    else
                        compteur <= compteur + 1;
                    end if;

                when PHASE_3 =>
                    if compteur < 7 then
                        compteur <= compteur + 1;
                    elsif BTN_G = '1' then
                        etat_courant <= PHASE_4;
                        compteur <= 0;
                    end if;

                when PHASE_4 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_1;
                        compteur <= 0;
                    else
                        compteur <= compteur + 1;
                    end if;
            end case;
        end if;
    end process;
```

```vhdl
process(etat_courant)
begin
    case etat_courant is
        when PHASE_1 =>
            LED_3_0 <= "0001";
            LED_7_4 <= "0100";
        when PHASE_2 =>
            LED_3_0 <= "0010";
            LED_7_4 <= "0100";
        when PHASE_3 =>
            LED_3_0 <= "0100";
            LED_7_4 <= "0001";
        when PHASE_4 =>
            LED_3_0 <= "0100";
            LED_7_4 <= "0010";
    end case;
end process;
```

Figure 3.1 : Code VHDL avec capteurs intégrés

3. Testez votre modèle en simulation.

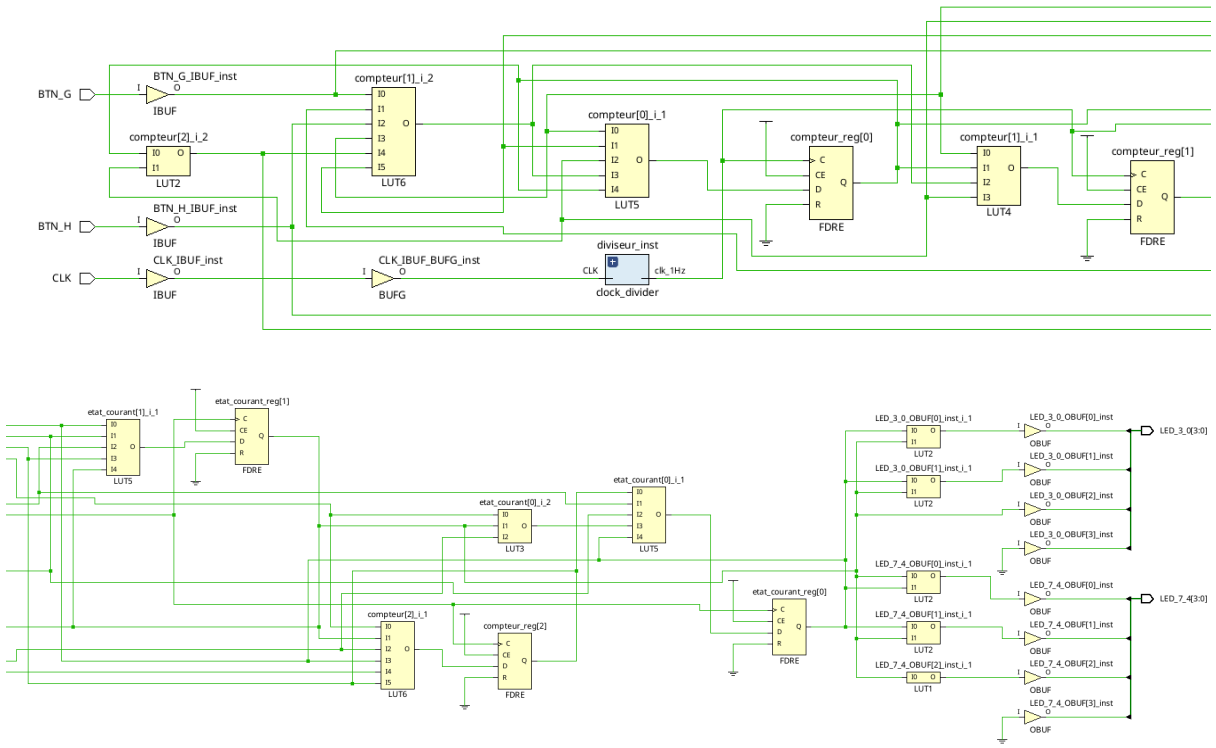


Figure 3.2 : Schéma de synthèse

Voici les scénarios précis que tu dois forcer pour prouver que ton code fonctionne :

- **Scénario 1 : Personne au carrefour (L'état de repos)**
  - **Force BTN_G à 0 et BTN_H à 0 en continu.**

- ***Ce que tu dois observer :*** **Ton système démarre en Phase 1 (Axe 1 Vert, Axe 2 Rouge) et y reste indéfiniment, même après que ton compteur ait atteint 7 (soit 8 secondes écoulées).**

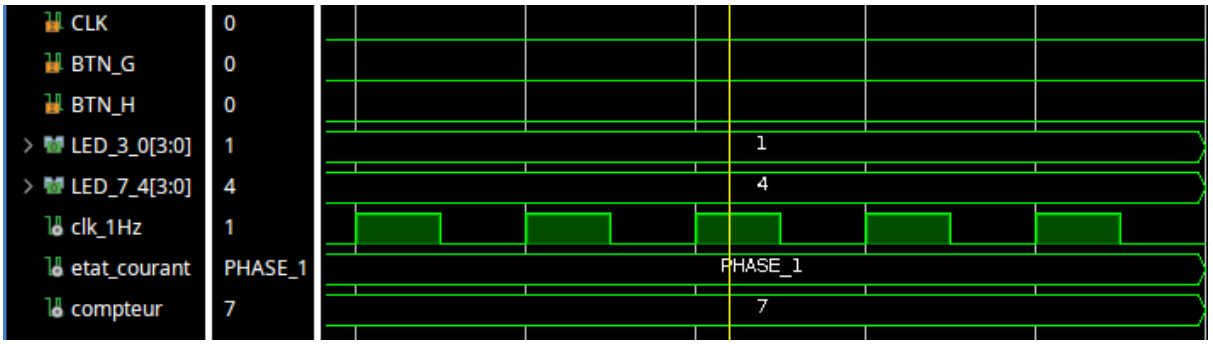


Figure 3.3 : Simulation pour la situation 1

- **Scénario 2 : Une voiture arrive sur l'axe 2**
  - **Laisse le système tourner un peu en Phase 1, puis force BTN_H à 1.**
  - ***Ce que tu dois observer :*** **Si le feu vert de l'axe 1 était allumé depuis *moins* de 8 secondes, le système doit attendre que ce temps soit écoulé avant de changer. Une fois le temps réglementaire respecté, il passe en Phase 2 (Orange).**

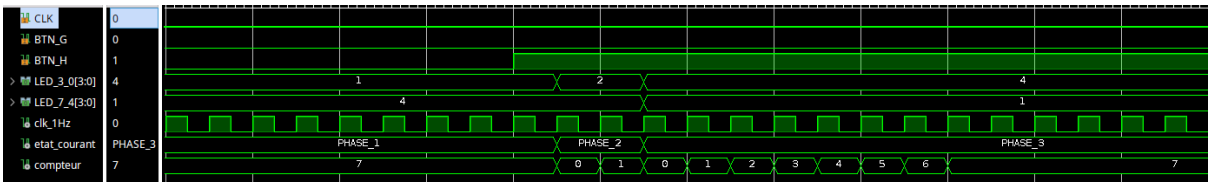


Figure 3.4 : Simulation pour la situation 2

- **Scénario 3 : Le retour à la normale (Une voiture arrive sur l'axe 1)**
  - **Attends que le système atteigne la Phase 3 (Axe 1 Rouge, Axe 2 Vert). Mets BTN_H à 0 et force BTN_G à 1.**
  - ***Ce que tu dois observer :*** **Comme pour l'autre axe, le système attend la fin des 8 secondes minimales, puis passe en Phase 4 (Orange) avant de revenir à la Phase 1.**

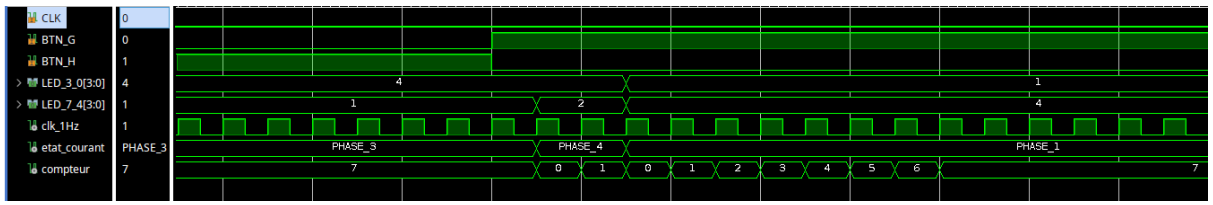


Figure 3.5: Simulation pour la situation 3

4. Programmez le FPGA et vérifiez le fonctionnement.

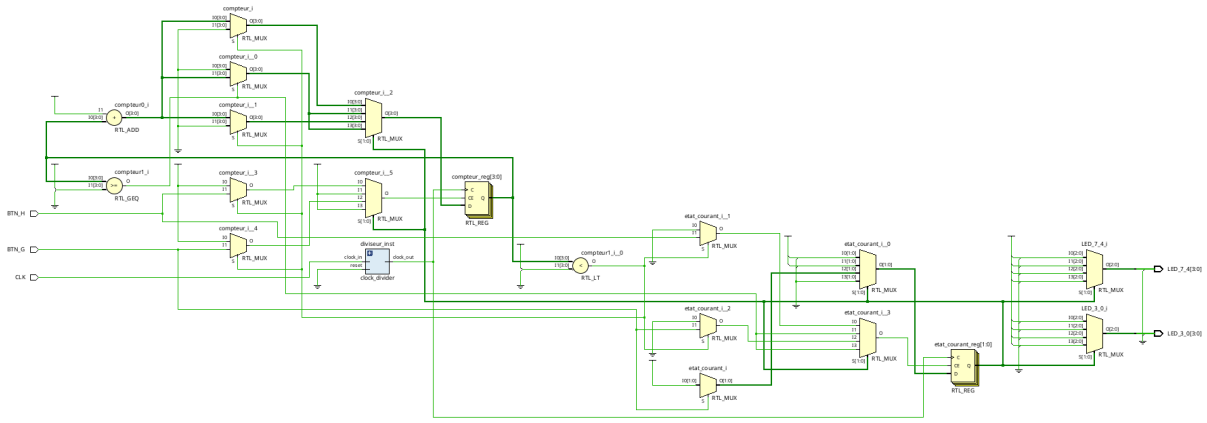


Figure 3.6 : Schéma RTL pré-synthèse

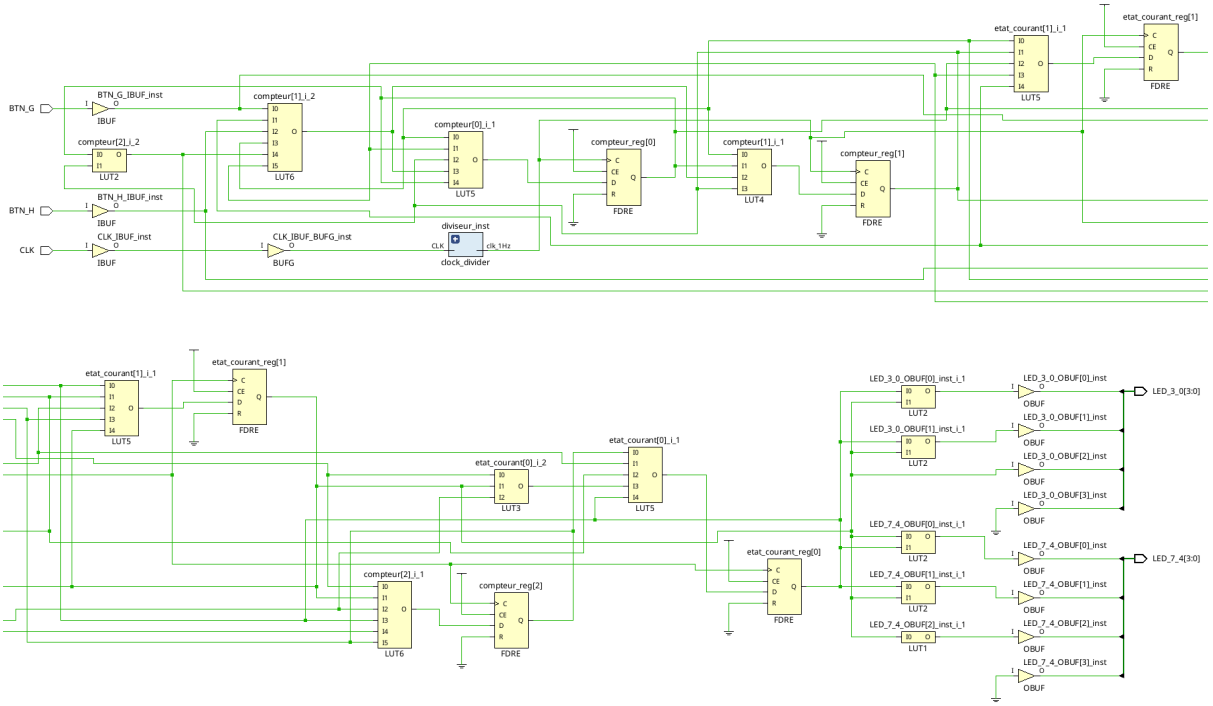


Figure 3.7 : Schéma de synthèse

Sur la carte, sans appui sur les boutons, le feu de l'axe 1 reste vert indéfiniment. Lorsqu'on appuie sur BTN_H, l'axe 1 passe à l'orange puis l'axe 2 au vert. L'axe 2 reste au vert jusqu'à ce qu'on appuie sur BTN_G, ce qui valide le fonctionnement matériel asservi aux capteurs.

5. Quel type de reset (synchrone ou asynchrone) vaut-il mieux mettre en œuvre et à quel moment doit-il être pris en compte ?

Un reset synchrone est préférable. Les capteurs étant des signaux asynchrones extérieurs, synchroniser le reset sur le front montant de clk_1Hz évite les aléas logiques et garantit une réinitialisation propre de la machine à états et du compteur.

6. Implémentez le reset, programmez le FPGA et testez le fonctionnement.

On intègre la condition if BTN_C = '1' en tête du process(clk_1Hz) pour forcer PHASE_1 et compteur <= 0. Sur la carte, un appui prolongé sur BTN_C réinitialise instantanément le carrefour au vert sur l'axe 1, ignorant les demandes en cours des capteurs et validant le montage final.

```
process(clk_1Hz)
begin
    if clk_1Hz'event and clk_1Hz = '1' then
        if BTN_C = '1' then
            etat_courant <= PHASE_1;
            compteur <= 0;
        else
            case etat_courant is
                when PHASE_1 =>
                    if compteur < 7 then
                        compteur <= compteur + 1;
                    elsif BTN_H = '1' then
                        etat_courant <= PHASE_2;
                        compteur <= 0;
                    end if;

                when PHASE_2 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_3;
                        compteur <= 0;
                    else
                        compteur <= compteur + 1;
                    end if;

                when PHASE_3 =>
                    if compteur < 7 then
                        compteur <= compteur + 1;
                    elsif BTN_G = '1' then
                        etat_courant <= PHASE_4;
                        compteur <= 0;
                    end if;

                when PHASE_4 =>
                    if compteur >= 1 then
                        etat_courant <= PHASE_1;
                        compteur <= 0;
                    else
                        compteur <= compteur + 1;
                    end if;
            end case;
        end if;
    end if;
end process;
```

Figure 3.8 : Modification du code VHDL avec reset

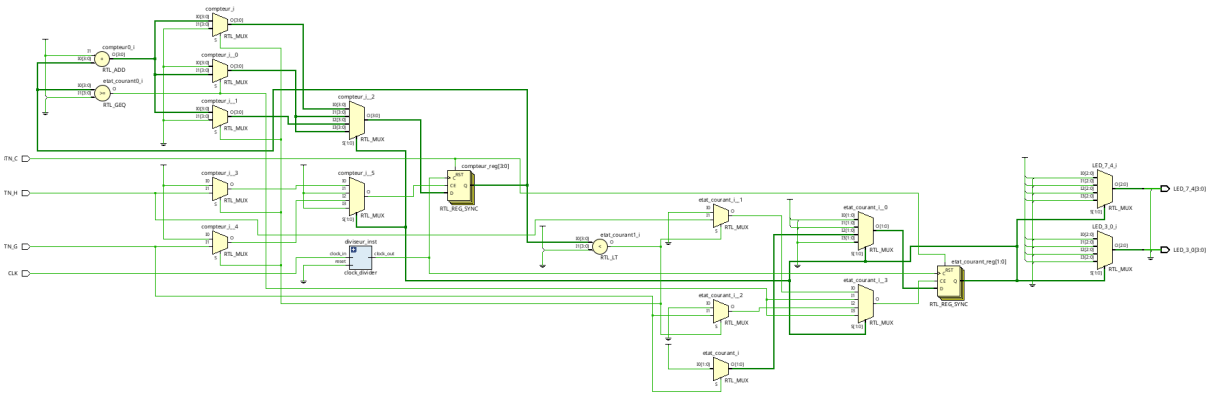


Figure 3.9 : Schéma RTL pre-synthèse

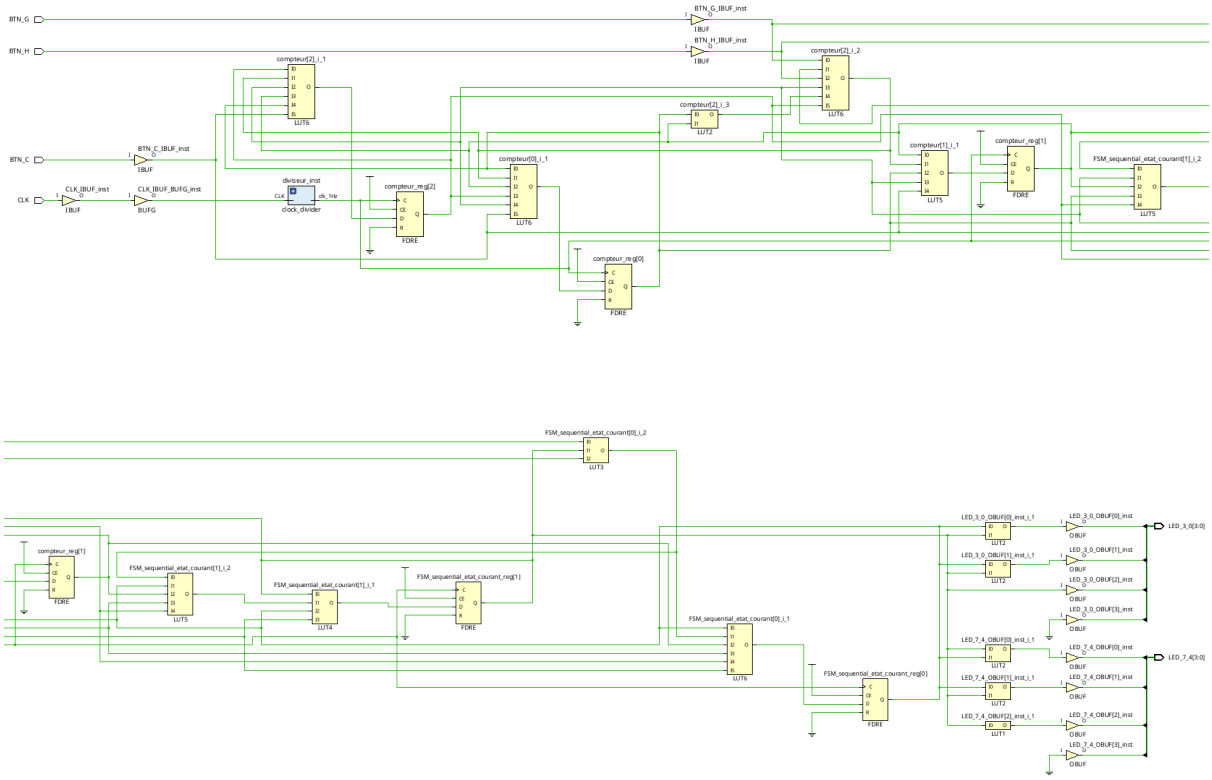


Figure 3.10 : Schéma de synthèse

Après programmation du FPGA, on teste la prise en compte du bouton BTN_C pendant le fonctionnement asservi aux capteurs. Quel que soit l'état actif du carrefour, un appui prolongé d'au moins une seconde sur BTN_C force immédiatement le retour à la phase 1 avec l'axe 1 au vert et l'axe 2 au rouge, tout en réinitialisant le compteur à zéro. Le système reste figé dans cet état tant que le bouton est maintenu, puis redémarre pour une durée verte complète de 8 secondes sur l'axe 1 dès qu'il est relâché avant de recommencer à écouter les demandes sur BTN_H et BTN_G.